We built a mini Docker-like engine called *Docksmith*.

Flow:

Docksmithfile → build → layers → image → run → isolated container

We built Docksmith, a mini container engine that works like a simplified Docker: it reads a Docksmithfile, converts each instruction into immutable tar-based layers identified by SHA-256 hashes, and uses a deterministic caching system to avoid rebuilding unchanged steps. During execution, it reconstructs a filesystem by extracting a minimal Alpine Linux base image and applying all layers, then enforces OS-level isolation using `chroot`, ensuring any process runs inside a confined environment. This is demonstrated through a cybersecurity test where a script attempts to write to `/etc` but is safely contained within the sandbox, proving that our system achieves secure, isolated execution with reproducible builds.

1. Created Project Structure

We made:

```
~/.docksmith/
  base/      → Alpine OS
  layers/    → tar files (our filesystem changes)
  cache/     → cache keys
  tmp/       → runtime container

docksmith.py  → main engine
Docksmithfile → instructions
script.py     → demo attack
```

Meaning:

- We separated **engine logic** from **data storage**
- Just like Docker does internally

2. Docksmithfile

Example:

```
FROM alpine
WORKDIR /app
COPY script.py .
CMD ["python3", "script.py"]
```

This is like:

“How to build my container”

3. Parsing the File

parse_docksmithfile()

What it does:

- Reads each line
- Splits into:

  ("COPY", "script.py .")

Output:

```
[
("FROM", "alpine"),
("COPY", "script.py ."),
("CMD", ...)
]
```

Meaning:
We convert text → structured instructions

4. Layer System

create_layer(src)

What happens:

1. Take file (`script.py`)
2. Put it into a `.tar` archive
3. Hash it using SHA-256
4. Store as:

~/.docksmith/layers/<hash>.tar

WHY LAYERS?

Instead of copying whole filesystem:

- We store only changes

Just like Docker:

Layer1 → base
Layer2 → COPY
Layer3 → RUN

5. Hashing

sha256_file()

Why?

- Same file → same hash
- Different file → different hash

This gives:
uniqueness
reproducibility
cache capability

6. Cache System

compute_cache_key(prev_hash, instruction)

Example:

prev_hash + "COPY script.py"

→ hashed → cache key

What happens:

First build:
[CACHE MISS]

Creates new layer

Second build:
[CACHE HIT]

Reuses old layer

Our build is deterministic and optimized

7. Build Process (Putting it together)

build()

For each instruction:

1. Compute cache key
2. Check cache
3. If miss → create layer
4. Store layer hash
5. Move to next step

Final output:

image_layers.txt
manifest.json

8. Manifest (Image Metadata)

```
{
 "layers": ["hash1", "hash2"],
 "cmd": "python3 /app/script.py"
}
```

Meaning:

- What layers to apply
- What command to run

This = Our image definition

9. Runtime

run()

This is where the magic happens.

Step 1: Create temp container

temp_dir = /tmp/xxxx

This becomes Our fake root filesystem

Step 2: Extract base OS

tar -xzf alpine.tar.gz

Now temp_dir contains:

/bin
/etc
/usr

We just created a **mini Linux OS**

Step 3: Apply layers

apply_layers()

Extract Our `.tar` layers on top

Now:

base OS + our files

Step 4: **chroot**os.chroot(temp_dir)

This changes:

/ → temp_dir

WHAT THIS MEANS

Inside container:

/etc → temp_dir/etc

NOT real system `/etc`

—----->This is isolation

10. Run command

subprocess.run(...)

Executes inside container

11. Cybersecurity Demo

OUr script:

with open("/etc/hacked.txt", "w") as f:

What happens:

Inside container:

/etc → temp_dir/etc

So file goes to:

/tmp/.../etc/hacked.txt

On host:

ls /etc/hacked.txt

NOT FOUND

FINAL PROOF

The attack was contained inside the container

COMPLETE FLOW

```
Docksmithfile
  ↓
Parse instructions
  ↓
Build layers (COPY, RUN)
  ↓
Store layers + cache
  ↓
Create manifest
  ↓
Run:
  → extract base OS
  → apply layers
  → chroot isolation
```

→ execute command

WHAT EACH PART PROVES

Feature	What it shows
Layers	Filesystem understanding
Hashing	Determinism
Cache	Optimization
chroot	OS-level isolation
Manifest	Image design
Demo	Security application

> We built a container engine using Linux primitives. Each instruction creates a hashed layer stored as a tar archive. During runtime, we reconstruct the filesystem by applying layers on a base image, then isolate execution using chroot. We also implemented deterministic caching using SHA-256, and demonstrated security by preventing malicious writes to the host system.

```
Mar 31 12:13
yuti@yuti-Legion-5-15IAX10: ~/.docksmith
yuti@yuti-Legion-5-15IAX10: $ mkdir -p ~/.docksmith/{images,layers,cache,tmp}
yuti@yuti-Legion-5-15IAX10: $ mkdir -p ~/.docksmith/base
cd ~/.docksmith/base
wget https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/alpine-minirootfs-3.19.0-x86_64.tar.gz
--2026-03-31 11:37:18--  https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/alpine-minirootfs-3.19.0-x86_64.tar.gz
Resolving dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)... 2a04:4e42:25::644, 151.101.158.132
Connecting to dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)|2a04:4e42:25::644|:443... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-03-31 11:37:19 ERROR 404: Not Found.

yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ wget https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/latest-releases.yaml
--2026-03-31 11:37:46--  https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/latest-releases.yaml
Resolving dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)... 2a04:4e42:25::644, 151.101.158.132
Connecting to dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)|2a04:4e42:25::644|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3320 (3.2K) [application/octet-stream]
Saving to: 'latest-releases.yaml'

latest-releases.yam 100%[===================>]   3.24K  --.-KB/s    in 0s

2026-03-31 11:37:47 (52.5 MB/s) - 'latest-releases.yaml' saved [3320/3320]

yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ touch docksmith.py
chmod +x docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ nano docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ nano docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ python3 docksmith.py build
Traceback (most recent call last):
  File "/home/yuti/.docksmith/base/docksmith.py", line 130, in <module>
    main()
  File "/home/yuti/.docksmith/base/docksmith.py", line 123, in main
    build()
  File "/home/yuti/.docksmith/base/docksmith.py", line 56, in build
    instructions = parse_docksmithfile()
                   ^^^^^^^^^^^^^^^^^^^^^
  File "/home/yuti/.docksmith/base/docksmith.py", line 17, in parse_docksmithfile
    with open("Docksmithfile") as f:
         ^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'Docksmithfile'
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ touch Docksmithfile
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ nano Docksmithfile
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ nano script.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ ^[[200~python3 docksmith.py build~^C
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ python3 docksmith.py build
[+] Layer created: 701c8e54778a5e664fc5a129d24b3e13059a407debb7a3e32d3a6180effce525
[✓] Build complete
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ ls
Docksmithfile  docksmith.py  image_layers.txt  latest-releases.yaml  script.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ nano ~/docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ cd ..
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano Docksmithfile
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano script.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ cd ~/docksmith
python3 docksmith.py build
bash: cd: /home/yuti/docksmith: No such file or directory
[+] Layer: 88b67f066bf479019fe465dc25228816d352acd6169f39eb1dfba35a1cae27e90
[✓] Build done
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ sudo python3 docksmith.py run
[sudo] password for yuti:
[*] Temp: /tmp/tmpfleniw2i
Traceback (most recent call last):
  File "/home/yuti/.docksmith/docksmith.py", line 117, in <module>
    main()
  File "/home/yuti/.docksmith/docksmith.py", line 112, in main
    run()
  File "/home/yuti/.docksmith/docksmith.py", line 83, in run
    subprocess.run(["tar", "-xzf", get_base(), "-C", root])
```

```
Mar 31 12:13
yuti@yuti-Legion-5-15IAX10: ~/.docksmith
                                  ^^^^^^^^^
  File "/home/yuti/.docksmith/docksmith.py", line 44, in get_base
    raise Exception("Base image missing")
Exception: Base image missing
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ ls ~/.docksmith/base
Docksmithfile  docksmith.py  image_layers.txt  latest-releases.yaml  script.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ rm -rf ~/.docksmith/base/*
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ cd ~/.docksmith/base
wget https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/alpine-minirootfs-latest-x86_64.tar.gz
--2026-03-31 11:47:29--  https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/alpine-minirootfs-latest-x86_64.tar.gz
Resolving dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)... 2a04:4e42:25::644, 151.101.210.132
Connecting to dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)|2a04:4e42:25::644|:443... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-03-31 11:47:29 ERROR 404: Not Found.

yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ wget https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/latest-releases.yaml
--2026-03-31 11:47:53--  https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/latest-releases.yaml
Resolving dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)... 2a04:4e42:25::644, 151.101.210.132
Connecting to dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)|2a04:4e42:25::644|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3320 (3.2K) [application/octet-stream]
Saving to: 'latest-releases.yaml'

latest-releases.yam 100%[===================>]   3.24K  --.-KB/s    in 0s

2026-03-31 11:47:53 (58.3 MB/s) - 'latest-releases.yaml' saved [3320/3320]

yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ cat latest-releases.yaml | grep minirootfs
  flavor: alpine-minirootfs
  file: alpine-minirootfs-3.23.3-x86_64.tar.gz
  iso: alpine-minirootfs-3.23.3-x86_64.tar.gz
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ wget https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/alpine-minirootfs-3.23.3-x86_64.tar.gz
--2026-03-31 11:48:22--  https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/alpine-minirootfs-3.23.3-x86_64.tar.gz
Resolving dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)... 2a04:4e42:31::644, 151.101.158.132
Connecting to dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)|2a04:4e42:31::644|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3713234 (3.5M) [application/octet-stream]
Saving to: 'alpine-minirootfs-3.23.3-x86_64.tar.gz'

alpine-minirootfs-3 100%[===================>]   3.54M  1.48MB/s    in 2.4s

2026-03-31 11:48:25 (1.48 MB/s) - 'alpine-minirootfs-3.23.3-x86_64.tar.gz' saved [3713234/3713234]

yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ ls
alpine-minirootfs-3.23.3-x86_64.tar.gz  latest-releases.yaml
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ cd ~/docksmith
sudo python3 docksmith.py run
bash: cd: /home/yuti/docksmith: No such file or directory
python3: can't open file '/home/yuti/.docksmith/base/docksmith.py': [Errno 2] No such file or directory
yuti@yuti-Legion-5-15IAX10:~/.docksmith/base$ cd ..
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ sudo python3 docksmith.py run
[*] Temp: /tmp/tmpibeznyśl
Traceback (most recent call last):
  File "/home/yuti/.docksmith/docksmith.py", line 117, in <module>
    main()
  File "/home/yuti/.docksmith/docksmith.py", line 112, in main
    run()
  File "/home/yuti/.docksmith/docksmith.py", line 83, in run
    subprocess.run(["tar", "-xzf", get_base(), "-C", root])
                   ^^^^^^^^^^^^^^^
  File "/home/yuti/.docksmith/docksmith.py", line 44, in get_base
    raise Exception("Base image missing")
Exception: Base image missing
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ find - -name "docksmith.py"
find: '/home/yuti/misp-docker/configs': Permission denied
find: '/home/yuti/misp-docker/logs': Permission denied
find: '/home/yuti/misp-docker/files': Permission denied
find: '/home/yuti/misp-docker/gnupg': Permission denied
find: '/home/yuti/wazuh-docker/single-node/config/wazuh_indexer_ssl_certs': Permission denied
```

Mai 31 12:14

yuti@yuti-Legion-5-15IAX10: ~/.docksmith

```
  File "/home/yuti/.docksmith/docksmith.py", line 83, in run
    subprocess.run(["tar", "-xzf", get_base(), "-C", root])
                                   ^^^^^^^^^^
  File "/home/yuti/.docksmith/docksmith.py", line 44, in get_base
    raise Exception("Base image missing")
Exception: Base image missing
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ find ~ -name "docksmith.py"
find: '/home/yuti/misp-docker/configs': Permission denied
find: '/home/yuti/misp-docker/logs': Permission denied
find: '/home/yuti/misp-docker/files': Permission denied
find: '/home/yuti/misp-docker/gnupg': Permission denied
find: '/home/yuti/wazuh-docker/single-node/config/wazuh_indexer_ssl_certs': Permission denied
/home/yuti/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ sudo python3 docksmith.py run
[*] Temp: /tmp/tmpuj_vfqk6
Traceback (most recent call last):
  File "/home/yuti/.docksmith/docksmith.py", line 117, in <module>
    main()
  File "/home/yuti/.docksmith/docksmith.py", line 112, in main
    run()
  File "/home/yuti/.docksmith/docksmith.py", line 83, in run
    subprocess.run(["tar", "-xzf", get_base(), "-C", root])
                                   ^^^^^^^^^^
  File "/home/yuti/.docksmith/docksmith.py", line 44, in get_base
    raise Exception("Base image missing")
Exception: Base image missing
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ ls ~/.docksmith/base
alpine-minirootfs-3.23.3-x86_64.tar.gz  latest-releases.yaml
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ sudo python3 docksmith.py run
[*] Temp: /tmp/tmp6qqnlzqh
Traceback (most recent call last):
  File "/home/yuti/.docksmith/docksmith.py", line 119, in <module>
    main()
  File "/home/yuti/.docksmith/docksmith.py", line 114, in main
    run()
  File "/home/yuti/.docksmith/docksmith.py", line 85, in run
    subprocess.run(["tar", "-xzf", get_base(), "-C", root])
                                   ^^^^^^^^^^
  File "/home/yuti/.docksmith/docksmith.py", line 44, in get_base
    for file in os.listdir(base_path):
                ^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: '/root/.docksmith/base'
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ sudo python3 docksmith.py run
[*] Temp: /tmp/tmpsu_o8jt4
[*] Inside container
/bin/sh: python3: not found
fallback works
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ python3 docksmith.py build
[CACHE MISS] FROM alpine
[CACHE MISS] WORKDIR /app
[CACHE MISS] COPY script.py .
[CACHE MISS] CMD ["python3", "script.py"]
[✓] Build complete
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ python3 docksmith.py build
[CACHE MISS] FROM alpine
[CACHE MISS] WORKDIR /app
[CACHE HIT] COPY script.py .
[CACHE MISS] CMD ["python3", "script.py"]
[✓] Build complete
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ sudo python3 docksmith.py run
[*] Temp: /tmp/tmp5tgk_hmc
[*] Inside container
/bin/sh: python3: not found
```

Mai 31 12:14

yuti@yuti-Legion-5-15IAX10: ~/.docksmith


```
[*] Temp: /tmp/tmpuj_vfqk6
Traceback (most recent call last):
  File "/home/yuti/.docksmith/docksmith.py", line 117, in <module>
    main()
  File "/home/yuti/.docksmith/docksmith.py", line 112, in main
    run()
  File "/home/yuti/.docksmith/docksmith.py", line 83, in run
    subprocess.run(["tar", "-xzf", get_base(), "-C", root])
                                   ^^^^^^^^^^
  File "/home/yuti/.docksmith/docksmith.py", line 44, in get_base
    raise Exception("Base image missing")
Exception: Base image missing
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ ls ~/.docksmith/base
alpine-minirootfs-3.23.3-x86_64.tar.gz  latest-releases.yaml
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ sudo python3 docksmith.py run
[*] Temp: /tmp/tmp6qqnlzqh
Traceback (most recent call last):
  File "/home/yuti/.docksmith/docksmith.py", line 119, in <module>
    main()
  File "/home/yuti/.docksmith/docksmith.py", line 114, in main
    run()
  File "/home/yuti/.docksmith/docksmith.py", line 85, in run
    subprocess.run(["tar", "-xzf", get_base(), "-C", root])
                                   ^^^^^^^^^^
  File "/home/yuti/.docksmith/docksmith.py", line 44, in get_base
    for file in os.listdir(base_path):
                ^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: '/root/.docksmith/base'
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ sudo python3 docksmith.py run
[*] Temp: /tmp/tmpsu_o8jt4
[*] Inside container
/bin/sh: python3: not found
fallback works
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ python3 docksmith.py build
[CACHE MISS] FROM alpine
[CACHE MISS] WORKDIR /app
[CACHE MISS] COPY script.py .
[CACHE MISS] CMD ["python3", "script.py"]
[✓] Build complete
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ python3 docksmith.py build
[CACHE MISS] FROM alpine
[CACHE MISS] WORKDIR /app
[CACHE HIT] COPY script.py .
[CACHE MISS] CMD ["python3", "script.py"]
[✓] Build complete
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ sudo python3 docksmith.py run
[*] Temp: /tmp/tmp5tgk_hmc
[*] Inside container
/bin/sh: python3: not found
fallback works
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ nano ~/.docksmith/docksmith.py
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ python3 docksmith.py build
[CACHE MISS] FROM alpine
[CACHE MISS] WORKDIR /app
[CACHE HIT] COPY script.py .
[CACHE MISS] CMD ["python3", "script.py"]
[✓] Build complete
yuti@yuti-Legion-5-15IAX10:~/.docksmith$ sudo python3 docksmith.py run
```


```
[*] Temp: /tmp/tmpvenht84_
[*] Inside container
/bin/sh: python3: not found
fallback works
yuti@yuti-Legion-5-15IAX10:~/.docksmith$
```

alpine-minirootfs-3.23.3-x86_64.tar.gz

Meaning:

- This is the actual OS image name
- We will use this as our container base

wget alpine-minirootfs-3.23.3-x86_64.tar.gz

WE downloaded a minimal Linux filesystem

This becomes our container OS

What this means

CACHE MISS

- No previous build exists
- Every step is executed fresh

Each instruction explained

1. `FROM alpine`

- Sets base image
- (we don't process it deeply, but it defines starting point)

2. `WORKDIR /app`

- Intended working directory
- (currently minimal handling in our code)

3. `COPY script.py .`

What happened:

- our file was:
    - archived into `.tar`
    - hashed (SHA-256)
    - stored in:

      ~/.docksmith/layers/<hash>.tar

This creates a layer

4. `CMD ["python3", "script.py"]`

- Stored in manifest
- Defines what runs inside container

Command

python3 docksmith.py build

Output

[CACHE MISS] FROM alpine

[CACHE MISS] WORKDIR /app

[CACHE HIT] COPY script.py .

[CACHE MISS] CMD ["python3", "script.py"]

[✓] Build complete

[CACHE HIT] COPY script.py .

Meaning:

- The file didn't change
- So:

    - layer was reused
    - no rebuild happened

Why this is important

This proves:

> our system is deterministic and optimized

VERY HIGH VALUE feature

`[*] Temp: /tmp/tmpXXXX`

Meaning:

- A temporary directory was created
- This is our container filesystem

What happened internally

1. Alpine base extracted
2. Layers applied
3. Filesystem built

WE now have a complete isolated OS

`[*] Inside container`

Meaning:

- `chroot()` executed
- Root directory changed to temp folder

Before:

/ → real system

After:

/ → /tmp/tmpXXXX

We are now inside a fake root

`/bin/sh: python3: not found`

Meaning:

- Alpine doesn't have Python installed
- So script didn't execute fully

`fallback works`

Meaning:

- our fallback command executed
- Container execution succeeded

Command

sudo python3 docksmith.py run

Output

[*] Temp: /tmp/tmpXXXXX

[*] Inside container

/bin/sh: python3: not found

fallback works

- Everything works consistently
- Build + cache + runtime stable

What really happened during run

our script tried:

open("/etc/hacked.txt", "w")

Inside container:

/etc → /tmp/tmpXXXXX/etc

File is created here

On host:

ls /etc/hacked.txt

Not found

PROOF

our system prevented a malicious write to host OS